UNIVERSITÀ
DEGLI STUDI DI BARI
ALDO MORO

COMPUTER SCIENCE DEPARTMENT

Computer Science - Curriculum Artificial Intelligence

Project Assignment

Formal Methods

Process Mining

Student: *Fontana Emanuele*

Academic Year 2024/2025

Indice

1 Introduction to Process Mining

1.1 Petri Nets

Petri nets are a mathematical and graphical model used to represent distributed and concurrent systems. In this model, systems are represented as networks composed of nodes and directed arcs. The nodes represent the states of the system, while the arcs represent transitions between these states. Petri nets are widely used in computer science to model complex systems, such as parallel processing systems, production control systems, and data processing systems.

Components of Petri Nets

- **Places**:
  - Represent states or conditions within the system.
  - Typically depicted as circles.
  - A place can hold tokens that represent the system's current state.
- **Transitions**:
  - Represent events or actions that cause changes in the system.
  - Typically depicted as rectangles or bars.
  - A transition occurs (fires) when there are sufficient tokens in the connected places to satisfy the preconditions.
- **Tokens**:
  - Represent the dynamic state of the system at any given time.
  - Tokens are placed in the places and can move through the network as transitions fire.

1.2 Process Mining

Process mining is a discipline that uses data analysis techniques to improve the understanding and optimization of business processes. It relies on the digital recording of data related to business processes, which is then analyzed to identify opportunities for improvement and optimization. Process mining includes several techniques, such as process visualization, analysis of wait times and durations, automatic discovery of process models, and analysis of deviations from established models. The end result is a deeper understanding of business processes, which can

be used to implement solutions for their optimization. Process mining is a valuable technology for many companies and organizations, as it helps identify inefficiencies in processes, reduce waiting times, and improve service quality to customers. Furthermore, process mining offers an intuitive visualization of business processes, which facilitates communication and collaboration among different stakeholders.

A process consists of actions (possibly concurrent) performed by agents (human or artifacts). Its dynamic behavior can be characterized in terms of events referring to identifiable actions (including decisions on the next task to be performed). Atomic activities are represented as single events, while activities that span a significant amount of time are represented by the interval between the start and end of the corresponding events.

1.3 Directly-Follows Graph (DFG)

The **Directly-Follows Graph (DFG)** is a graphical representation used in *process mining* to describe the direct relationships between activities in a process. In this graph:

- The **nodes** represent the activities (or events) that occur in the process.
- The **edges** represent the direct "follows" relationship, meaning that one activity immediately follows another in the event log.
- The **direction of the edges** indicates the sequence of execution, with an edge from Activity A to Activity B meaning that Activity B directly follows Activity A.
- The **edge weight** often represents the frequency with which one activity follows another.

The DFG helps in visualizing the flow of the process and is often used as a foundation for process discovery algorithms, such as Alpha Miner, Inductive Miner, and Heuristic Miner.

1.3.1 Alpha Miner

The **Alpha Miner algorithm** is a foundational technique in *process mining* used to discover process models from event logs. It reconstructs a workflow model, typically represented as a Petri net, based on the sequence and concurrency of events. Below is a brief explanation:

Key Concepts

- **Event Log**: A collection of traces, where each trace is a sequence of events (activities) performed during a process instance.
- **Causal Relations**: The algorithm identifies relationships between activities:
  - *Direct causality* ($A \rightarrow B$): Activity A directly precedes activity B in the log.
  - *Concurrency* ($A \parallel B$): Activities A and B can occur in any order, indicating parallel execution.
  - *No relation* ($A\#B$): No observable relationship between A and B.

Steps of the Alpha Miner Algorithm

1. **Preprocessing**: Parse the event log to extract traces and their activities.
2. **Identify Relations**:
   - Determine which activities are causally related, concurrent, or unrelated.
3. **Build a Petri Net**:
   - Create *places* (conditions) to represent causal dependencies.
   - Add *transitions* for each activity and connect them via places based on the identified relations.
4. **Generate Initial and Final Places**:
   - Define the starting and ending conditions of the process.

1.3.2 Inductive Miner Algorithm

The **Inductive Miner algorithm** is a popular technique in *process mining* that generates process models from event logs. Unlike the Alpha Miner, it guarantees the creation of sound workflow models by recursively dividing and conquering the event log data. Below is an overview of the algorithm:

Key Concepts

- **Event Log**: A collection of traces where each trace represents a sequence of activities in a process instance.
- **Behavioral Patterns**: The algorithm identifies the following key patterns in the event log:
  - *Sequence*: Activities follow each other in a strict order.
  - *Concurrency*: Activities can occur in parallel.
  - *Choice*: One activity is chosen from several alternatives.
  - *Loops*: Activities repeat a certain number of times.

Steps of the Inductive Miner Algorithm

1. **Preprocessing**: Parse the event log and ensure it contains traces of activities with start and end points.
2. **Partitioning the Log**:
   - Divide the event log based on directly-follows relationships.
   - Identify patterns (e.g., sequence, concurrency, choice, or loop) based on how activities relate to one another.
3. **Recursive Construction**:
   - Create a submodel for each partition.
   - Combine submodels using operators (*sequence, concurrency, choice, loop*) to form the overall process model.
4. **Generate a Petri Net or Process Tree**:
   - Translate the resulting model into a Petri net or process tree for visualization and analysis.

1.3.3 Heuristic Miner Algorithm

The **Heuristic Miner algorithm** is a process discovery technique used in *process mining*. It extends the capabilities of the Alpha Miner by handling noise and incompleteness in event logs, making it more robust for real-world scenarios. The algorithm identifies causal relationships and constructs a dependency graph to represent the process.

Key Concepts

- **Event Log**: A collection of traces, where each trace is a sequence of events (activities) performed in a process instance.
- **Causal Dependency Measures**:
  - *Directly Follows Relation*: Determines if activity A is frequently followed by activity B in the log.
  - *Dependency Score*: Quantifies the strength of the causal relationship between activities based on their occurrence patterns.
- **Thresholds**: Used to filter out weak or noisy relations, focusing only on the most significant dependencies.
- **Dependency Graph**: A directed graph where nodes represent activities, and edges indicate dependencies, weighted by their strength.

Steps of the Heuristic Miner Algorithm

1. **Preprocessing**: Parse the event log to extract traces and their activities.
2. **Calculate Dependency Metrics**:
   - Compute the frequency of directly follows relations between pairs of activities.
   - Calculate the *dependency score* for each pair to determine the strength of their causal relationship.
3. **Filter Relations**:
   - Use predefined thresholds to remove weak or noisy relationships.
   - Retain only significant dependencies.
4. **Construct a Dependency Graph**:
   - Create a directed graph with nodes as activities and weighted edges as dependency scores.
5. **Generate a Process Model**:
   - Convert the dependency graph into a workflow model, such as a Petri net or a process tree.

Aspect	Alpha Miner	Inductive Miner	Heuristic Miner
Main Feature	Discovers simple process models	Guarantees sound models	Handles noise and incomplete event logs
Limitations	Sensitive to noise, struggles with complex loops	Limited to predefined patterns (e.g., sequence, loop)	Results depend on threshold settings

Tabella 1: Comparison of Alpha Miner, Inductive Miner, and Heuristic Miner algorithms.

1.3.4 Comparison

1.3.5 Evaluation Metrics

- **Fitness**: It evaluates whether the model can reproduce the behavior observed in the log.
- **Precision**: It indicates whether the model allows execution sequences that were not present in the log.
- **Simplicity**: It evaluates the complexity of the discovered process model. A simpler model is typically more understandable and easier to analyze.
- **Generalization**: It measures the ability of the discovered model to handle unseen behavior beyond

2 Dataset

2.1 Introduction to the dataset

Introduction to CASAS Dataset

In this study, we utilize a dataset from the Center for Advanced Studies in Adaptive Systems (CASAS). CASAS is a renowned research center focused on developing adaptive systems, particularly in the realm of smart environments and ubiquitous computing. The center provides a variety of datasets aimed at advancing the understanding and development of systems that interact intelligently with their surroundings.

The CASAS dataset contains data collected from sensor-equipped environments, typically used for applications such as activity recognition, healthcare monitoring, and smart home automation. These datasets often include information from sensors such as motion detectors, door sensors, temperature sensors, and other smart devices that track user behaviors and environmental conditions.

For process mining, CASAS datasets offer valuable insights into real-world human activities, allowing researchers to analyze patterns of behavior, optimize processes, and improve system performance. By using CASAS data, we can model and analyze the underlying processes, gaining a deeper understanding of complex behaviors in smart environments.

2.2 Preprocessing

Below is the Python code used for formatting a CSV file:

Listing 1: Format CSV Function

```
def format_csv(input_file, output_file):
    formatted_rows = []
    try:
        with open(input_file, 'r') as file:
            lines = file.readlines()
            formatted_rows.append("Timestamp, CaseID, Activity")
            for line in lines:
                line = line.strip()
                if not line: # Ignore empty lines
                    continue
                elements = line.split()
                if len(elements) >= 4:
                    timestamp = f"{elements[0]} {elements[1]}"
                    case_id = elements[2]
```

```
                activity = ' '.join(elements[3:])
                formatted_row = f"{timestamp}, {case_id}, {activity}"
                formatted_rows.append(formatted_row)
        with open(output_file, 'w', newline='') as file:
            for row in formatted_rows:
                file.write(row + '\n')
    except Exception as e:
        print(f"A problem occurred while formatting the CSV file: {e}")
```

The function `format_csv` reads a CSV file and formats it into a new CSV file with the columns `Timestamp`, `CaseID`, and `Activity`. The function extracts the timestamp, case ID, and activity from each line. It then writes the formatted rows to the output file.

Listing 2: CSV to XES Conversion

```
def csv_to_xes(input_file):
    data = pd.read_csv(input_file, sep=",")
    cols = ['time:timestamp', 'case:concept:name','concept:name']
    data.columns = cols
    data['time:timestamp'] = pd.to_datetime(data['time:timestamp'])
    data['concept:name'] = data['concept:name'].astype(str)
    log = log_converter.apply(data, variant=log_converter.Variants.TO
    output_name = input_file.split(".")[0] + ".xes"
    write_xes.write_xes(log, output_name)
```

The function `csv_to_xes` reads a CSV file and converts it into an XES file. It uses the `log_converter` and `write_xes` functions from the `pm4py` library to convert the CSV data into an event log. The event log is then written to an XES file with the same name as the input file.

3 Results

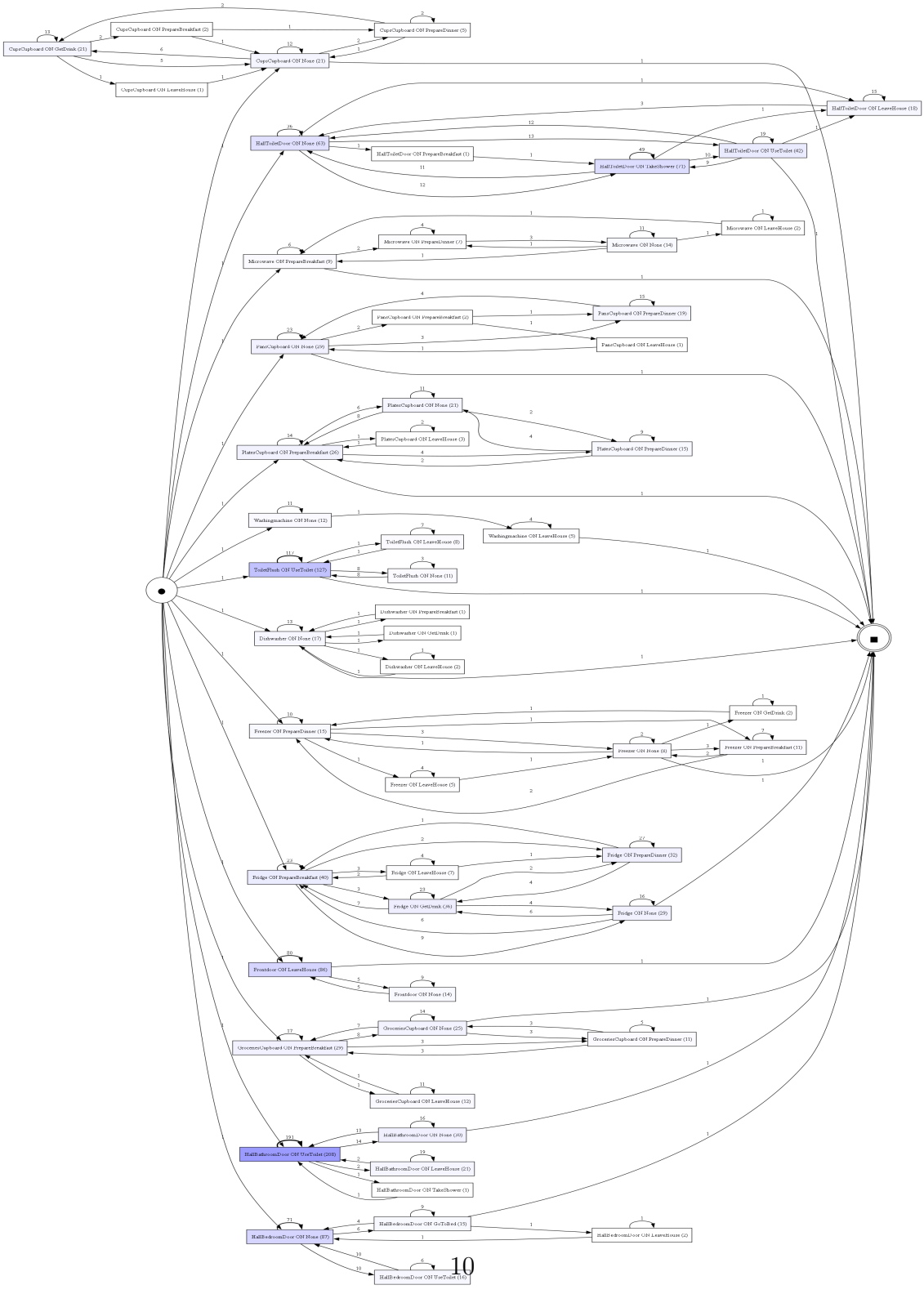

Figura 1: Directly-Follows Graph (DFG) of the event log

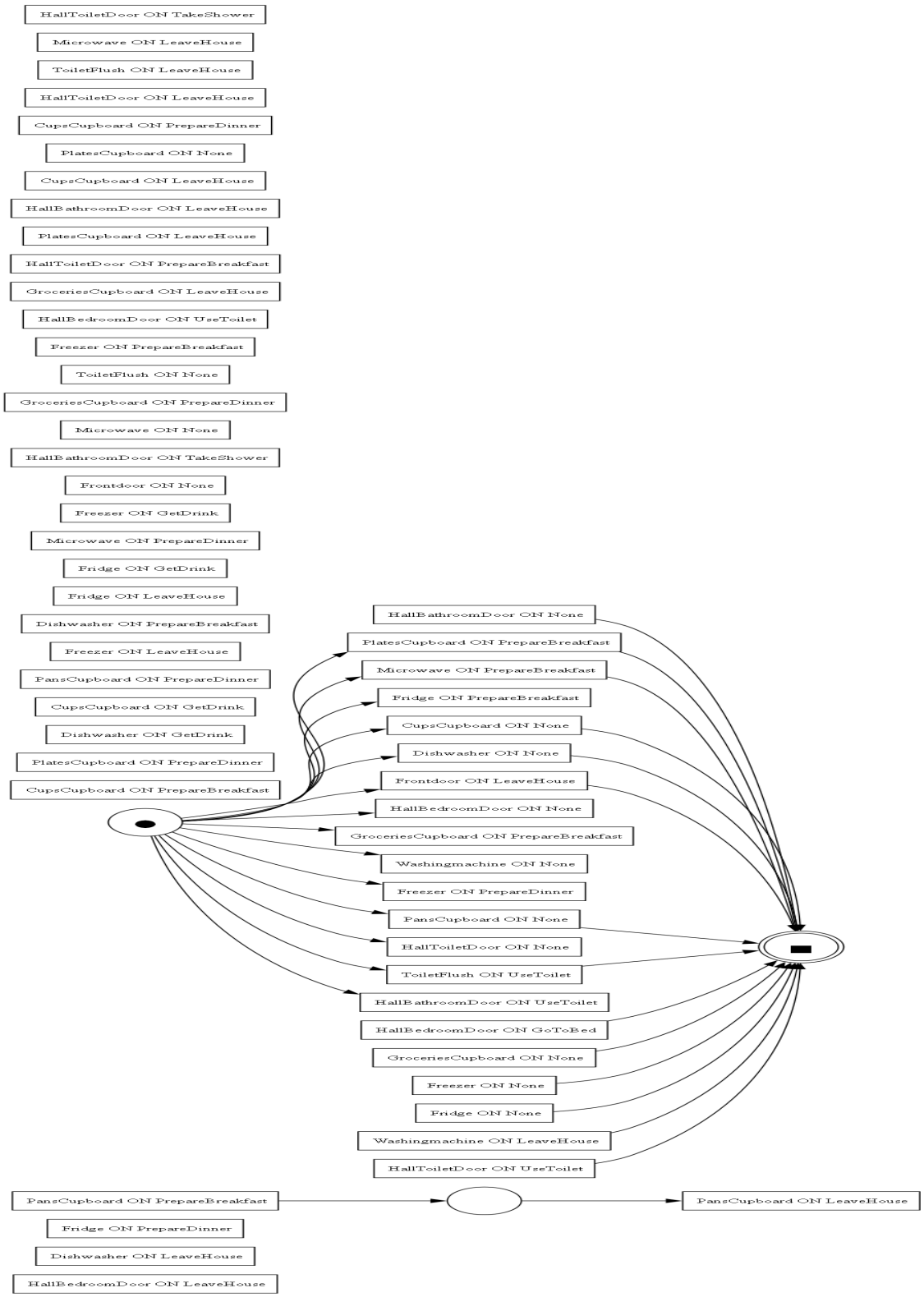

Figura 2: Alpha Miner Petri Net

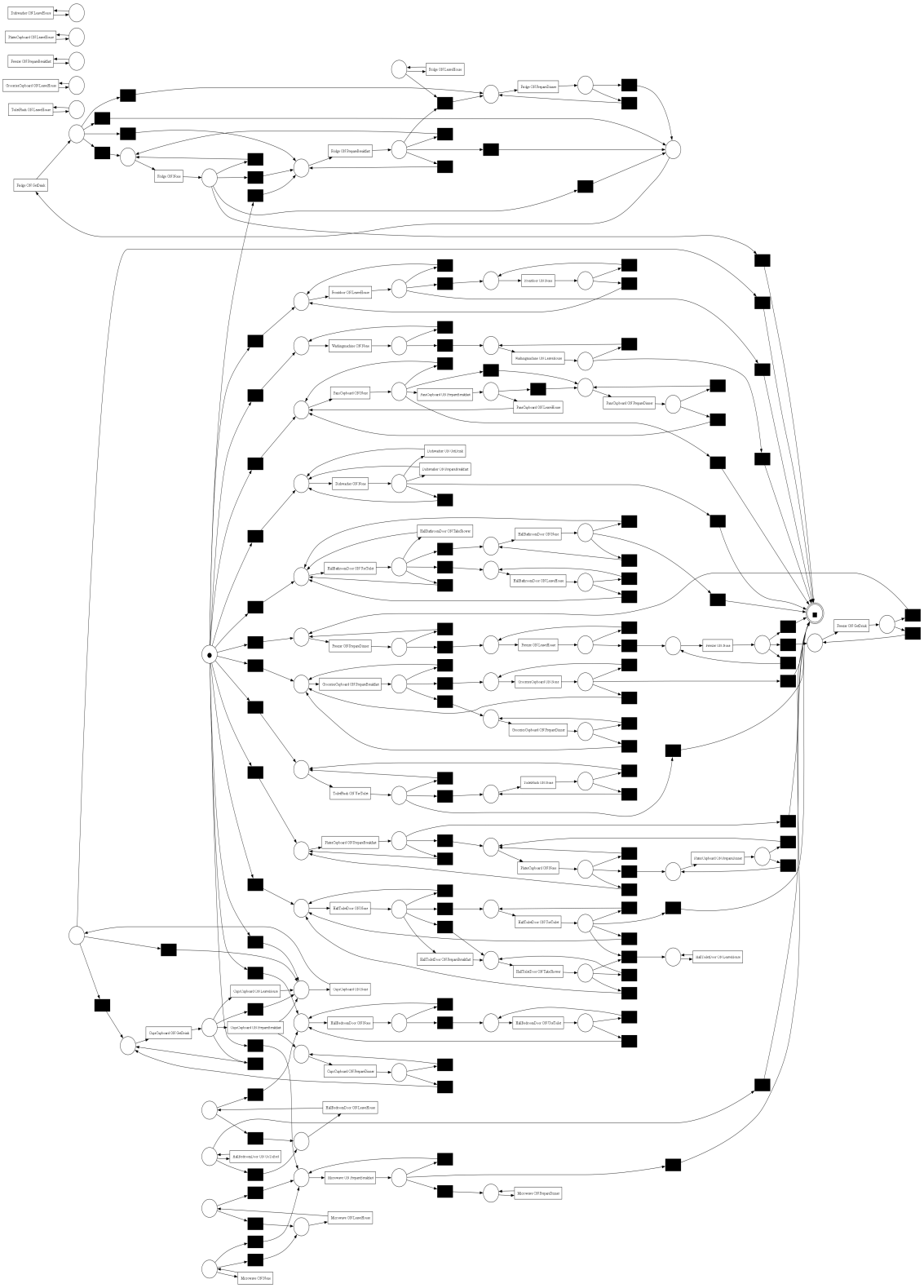

Figura 3: Heuristic Miner Petri Net

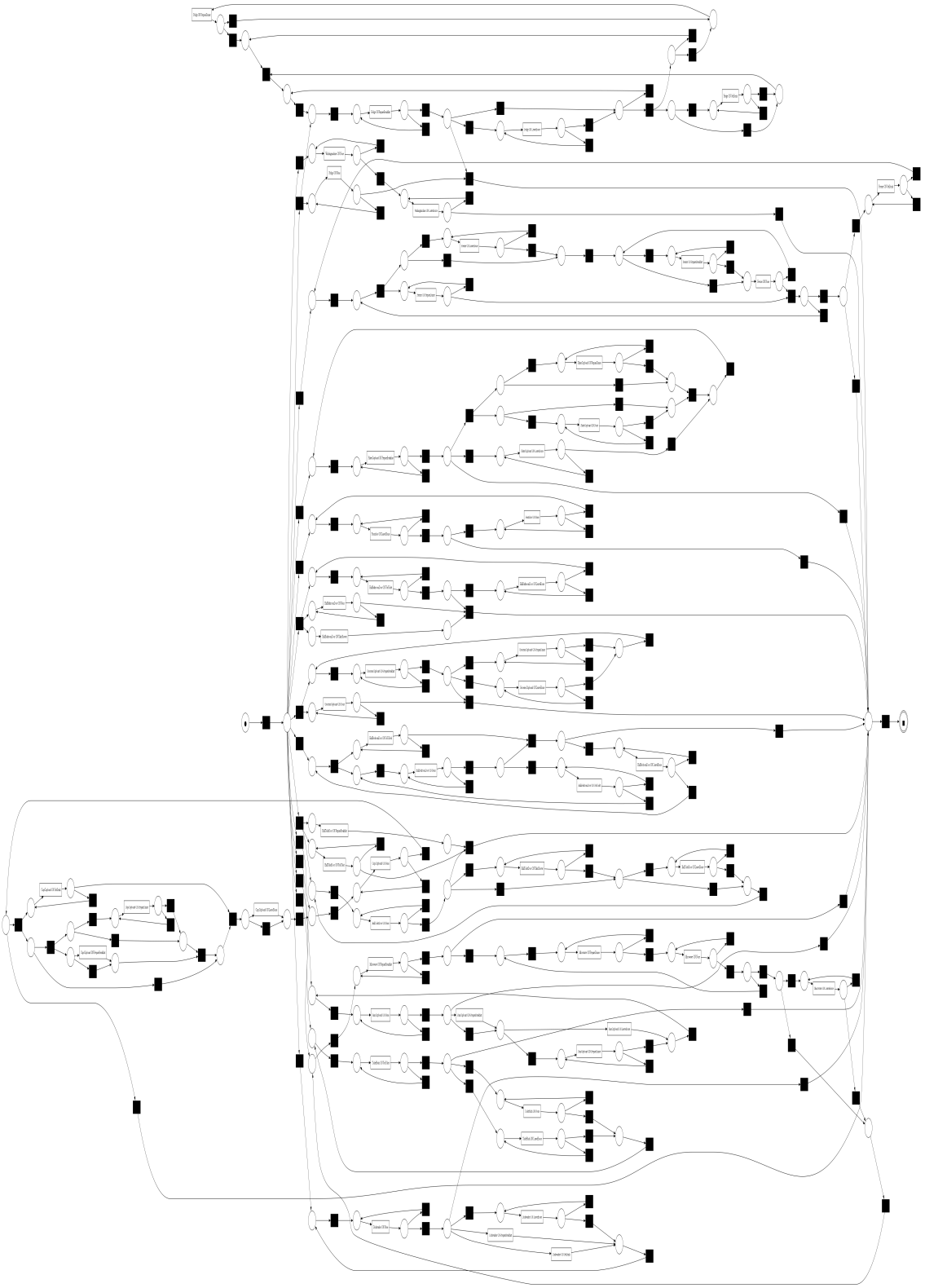

Figura 4: Inductive Miner Petri Net

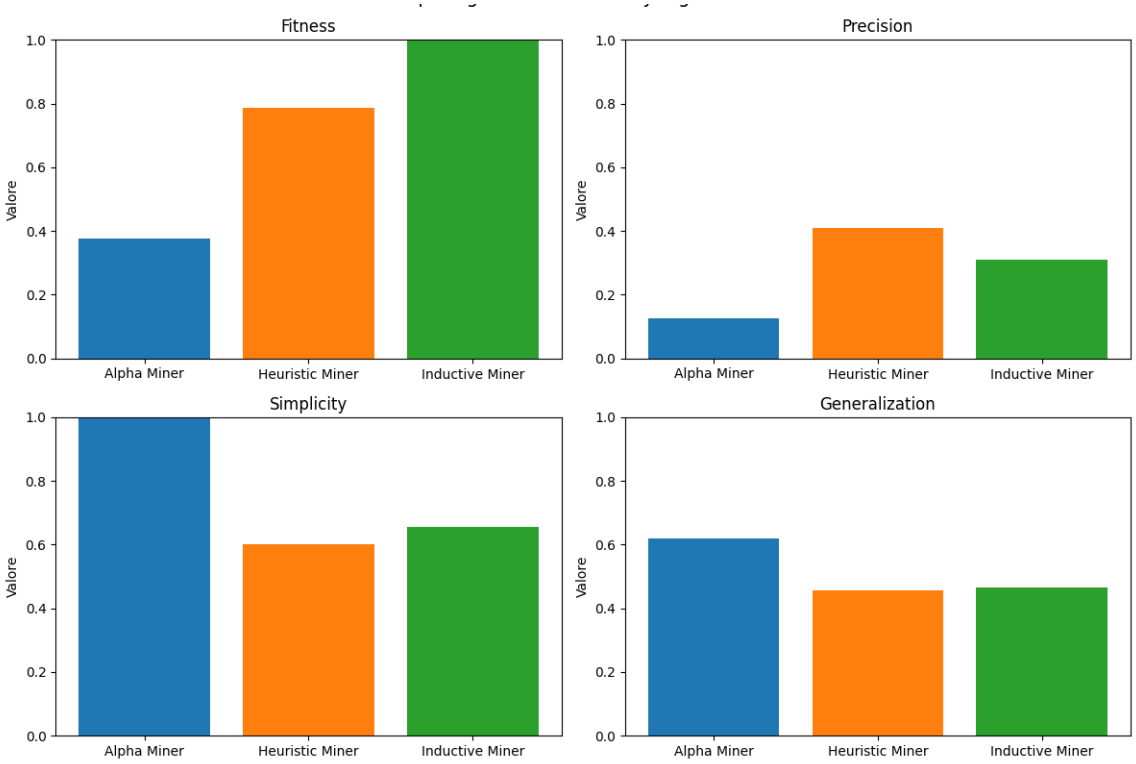

Figura 5: Performance metrics of the process mining algorithms

As we can see, the Alpha Miner algorithm produces a simple PetriNet and doesn't perform very well on the other metrics The Inductive Miner is probably the best algorithm for this dataset